115 ejercicios resueltos de Programación C++

Desde www.ra-ma.es podrás descargar material adicional.

JOFEBEUS
IRYOPOGU

Ra-Ma®
edU

Informática

115 ejercicios resueltos de programación C++

Jorge Fernando Betancourt Uscátegui
Irma Yolanda Polanco Guzmán

Ra-Ma®
edU®
Conocimiento a su alcance
Bogotá - México, D.F.

Betancourt Uscátegui, Jorge Fernando, *et al.*

115 ejercicios resueltos de programación C++/ Jorge Fernando Betancourt Uscátegui e Irma Yolanda Polanco Guzmán --. Bogotá: Ediciones de la U, 2021
518 p. ; 24 cm
ISBN 978-958-792-297-4 e-ISBN 978-958-792-298-1
1. Programación 2. Estructuras secuenciales 3. Cadenas 4. Almacenamiento Tít. 621.39 ed.

Edición original publicada por © Editorial Ra-ma (España)
Edición autorizada a Ediciones de la U para Colombia

Área: Informática
Primera edición: Bogotá, Colombia, septiembre de 2021
ISBN. 978-958-792-297-4

© Jorge Fernando Betancourt Uscátegui e Irma Yolanda Polanco Guzmán
© Ra-ma Editorial. Calle Jarama, 3-A (Polígono Industrial Igarsa) 28860 Paracuellos de Jarama
www.ra-ma.es y www.ra-ma.com / E-mail: editorial @ra-ma.com
Madrid, España
© Ediciones de la U - Carrera 27 #27-43 - Tel. (+57-1) 3203510 -3203499
www.edicionesdelau.com - E-mail: editor@edicionesdelau.com
Bogotá, Colombia

Ediciones de la U es una empresa editorial que, con una visión moderna y estratégica de las tecnologías, desarrolla, promueve, distribuye y comercializa contenidos, herramientas de formación, libros técnicos y profesionales, e-books, e-learning o aprendizaje en línea, realizados por autores con amplia experiencia en las diferentes áreas profesionales e investigativas, para brindar a nuestros usuarios soluciones útiles y prácticas que contribuyan al dominio de sus campos de trabajo y a su mejor desempeño en un mundo global, cambiante y cada vez más competitivo.

Coordinación editorial: Adriana Gutiérrez M.
Carátula: Ediciones de la U
Impresión: DGP Editores SAS
Calle 63 #70D-34, Pbx (57+1) 3203510

Impreso y hecho en Colombia
Printed and made in Colombia

No está permitida la reproducción total o parcial de este libro, ni su tratamiento informático, ni la transmisión de ninguna forma o por cualquier medio, ya sea electrónico, mecánico, por fotocopia, por registro y otros medios, sin el permiso previo y por escrito de los titulares del Copyright.

A nuestros padres por sus enseñanzas,
por los valores impartidos y por su
apoyo constante para la superación
tanto personal como profesional
A todos los interesados en acrecentar
su conocimiento en el campo de la
programación y la lógica en la solución
de problemas mediante el lenguaje C++

ÍNDICE

INTRODUCCIÓN **11**

CAPÍTULO 1. GENERALIDADES DEL LENGUAJE DE PROGRAMACIÓN C++ .. **15**

1.1 CARACTERÍSTICAS 15

1.2 ELEMENTOS BÁSICOS DE UN PROGRAMA 16

1.2.1 Instrucciones 16

1.2.2 Datos y tipos 16

1.2.3 Bibliotecas 17

1.2.4 Constantes simbólicas y constantes literales 18

1.2.5 Identificadores 19

1.2.6 Comentarios 19

1.2.7 Delimitadores 19

1.2.8 Operadores 19

1.2.9 Variables 20

1.2.10 Entrada / Salida 22

1.2.11 Asignación 23

1.2.12 Palabras reservadas 23

1.2.13 Secuencias de escape 24

1.2.14 Tabla ASCII 24

1.3 FASES DE LA EJECUCIÓN DE UN PROGRAMA 26

1.4 TIPOS DE ERRORES EN LA EJECUCIÓN DE UN PROGRAMA 26

1.4.1 Errores de tiempo de compilación 26

1.4.2 Errores de tiempo de enlazado 27

1.4.3 Errores de tiempo de ejecución (runtime) 28

1.5 EJEMPLO DE UN PROGRAMA 28

CAPÍTULO 2. ESTRUCTURAS SECUENCIALES **31**

2.1 SENTENCIA 31

2.2 ESTRUCTURA SECUENCIAL 31

2.3 EJERCICIOS RESUELTOS DE ESTRUCTURAS SECUENCIALES 32

CAPÍTULO 3. ESTRUCTURAS SELECTIVAS IF, IF – ELSE, IF ANIDADOS 53
3.1 ESTRUCTURA IF 53
3.2 ESTRUCTURA IF - ELSE 54
3.3 ESTRUCTURA IF – ELSE ANIDADA 56
3.4 EJERCICIOS RESUELTOS DE ESTRUCTURAS SELECTIVAS 58

CAPÍTULO 4. ESTRUCTURA SELECTIVA SWITCH 83
4.1 ESTRUCTURA SELECTIVA SWITCH 83
4.2 EJERCICIOS RESUELTOS DE ESTRUCTURA SELECTIVA SWITCH 86

CAPÍTULO 5. ESTRUCTURAS REPETITIVAS 145
5.1 ESTRUCTURA FOR 145
5.2 ESTRUCTURA WHILE 147
5.3 ESTRUCTURA DO - WHILE 149
5.4 EJERCICIOS RESUELTOS DE ESTRUCTURAS REPETITIVAS 150

CAPÍTULO 6. ARREGLOS UNIDIMENSIONALES (VECTORES) 175
6.1 CONCEPTO 175
6.2 DECLARACIÓN 175
6.3 INICIALIZAR ELEMENTOS DEL VECTOR 176
6.3.1 Inicialización de elementos del vector en su declaración 176
6.3.2 Inicialización de elementos del vector mediante asignación en forma aleatoria 177
6.3.3 Inicialización de elementos del vector mediante asignación por captura de datos 178
6.4 EJERCICIOS RESUELTOS DE VECTORES 178

CAPÍTULO 7. ARREGLOS MULTIDIMENSIONALES 203
7.1 CONCEPTO DE ARREGLOS MULTIDIMENSIONALES 203
7.2 DECLARACIÓN DE UNA MATRIZ O ARREGLO BIDIMENSIONAL 203
7.3 INICIALIZACIÓN DE LOS ELEMENTOS DE UNA MATRIZ O TABLA 204
7.3.1 Inicializar elementos de la tabla en su declaración 204
7.3.2 Inicializar elementos del arreglo mediante asignación en forma aleatoria 204
7.3.3 Inicializar elementos de la matriz mediante asignación por captura de datos 205
7.4 PROCESAMIENTO DE UN ARREGLO BIDIMENSIONAL O MATRIZ 206
7.5 EJERCICIOS RESUELTOS DE ARREGLOS BIDIMENSIONALES 207

CAPÍTULO 8. FUNCIONES DE USUARIO 243
8.1 DECLARACIÓN DE VARIABLES GLOBALES Y LOCALES 243
8.2 CONCEPTO DE PROCEDIMIENTO 244

8.3 CONCEPTO DE FUNCIÓN DE USUARIO 246
8.4 EJERCICIOS RESUELTOS DE FUNCIONES DE USUARIO 247

CAPÍTULO 9. RECURSIVIDAD 285
9.1 CONCEPTO 285
9.2 CARACTERÍSTICAS DE LA RECURSIVIDAD 286
9.3 TIPOS DE RECURSIVIDAD 286
9.4 VENTAJAS Y DESVENTAJAS DE LA RECURSIVIDAD 287
9.5 EJERCICIOS RESUELTOS APLICANDO RECURSIVIDAD 287

CAPÍTULO 10. CADENAS DE CARACTERES 313
10.1 CADENA DE CARACTERES 313
10.2 TIPO DE DATO STRING 314
10.3 ENTRADA Y SALIDA DE CADENAS DE CARACTERES 315
10.4 OPERACIONES PREDEFINIDAS 316
10.5 EJERCICIOS RESUELTOS DE CADENAS DE CARACTERES 318

CAPÍTULO 11. REGISTROS O ESTRUCTURAS 345
11.1 CONCEPTO DE REGISTRO O ESTRUCTURA 345
11.2 OPERACIONES CON ESTRUCTURAS COMPLETAS 347
11.2.1 Asignación 348
11.2.2 Paso como parámetro a procedimientos 348
11.2.3 Estructuras como valor de retorno de funciones 349
11.3 MIEMBROS COMO ESTRUCTURA 349
11.4 OPERACIONES CON ESTRUCTURAS 349
11.5 EJERCICIOS RESUELTOS DE ESTRUCTURAS 350

CAPÍTULO 12. ALMACENAMIENTO EN MEMORIA SECUNDARIA: FICHEROS O ARCHIVOS 385
12.1 ARCHIVOS DE TEXTO Y BINARIOS 386
12.2 FLUJOS DE ENTRADA Y SALIDA ASOCIADOS A ARCHIVOS 386
12.3 ENTRADA DE DATOS DESDE ARCHIVOS DE TEXTO 387
12.4 EJERCICIOS RESUELTOS DE MANEJO DE ARCHIVOS 389

CAPÍTULO 13. PROGRAMACIÓN ORIENTADA A OBJETOS POO 473
13.1 MÉTODOS Y ATRIBUTOS 473
13.2 DEFINICIÓN DE CLASES 473
13.3 OBJETOS 475
13.4 CONSTRUCTORES Y DESTRUCTORES 475
13.5 EJERCICIOS RESUELTOS DE CLASES 477

REFERENCIAS 515

MATERIAL ADICIONAL 517

INTRODUCCIÓN

El lenguaje de programación C++ se originó alrededor de la década de los 80 como una extensión del lenguaje C, en términos generales un programa en C++ consta de dos partes. La primera incluye las directivas del preprocesador en la que se especifican las librerías empleadas, la definición de variables globales y constantes y la segunda que corresponde al cuerpo del programa que se identifica con la función main y las llaves de inicio { y de fin } del bloque de instrucciones.

A continuación se describe el contenido de cada uno de los diferentes capítulos, donde al final de los mismos se presentan los ejercicios resueltos. En el capítulo 1 se introduce al lenguaje de programación C++, describe las principales características y generalidades, los elementos básicos de un programa como son las instrucciones o sentencias, los tipos de datos, las bibliotecas o librerías, la definición de constantes simbólicas y literales, el manejo y especificación de comentarios, los tipos de operadores empleados, la definición de variables y sus elementos, las sentencias de entrada y salida, de asignación, palabras reservadas, las sentencias de escape y la tabla ASCII. Además se describe las fases de ejecución de un programa, los tipos de errores y se presenta un ejemplo de un programa.

El capítulo 2 titulado Estructuras secuenciales, comienza con la definición del concepto de sentencia, la descripción de estructura secuencial como el bloque de sentencias ordenadas en forma lógica.

Las estructuras selectivas se describen en el capítulo 3. La toma de decisiones con base en la evaluación de condiciones se delimita mediante la estructura if, estructura if – else o cuando se requiera evaluar diferentes rangos se emplean las estructuras if anidadas.

En el capítulo 4 se describe otra estructura selectiva como lo es la estructura switch. La estructura selectiva switch es útil cuando se conoce con certeza el resultado de una expresión y de acuerdo con el mismo se ejecutan diversas acciones.

Las estructuras repetitivas, iterativas o cíclicas se describen en el capítulo 5. Es claro que cada estructura presenta sus características y se aplican en determinadas condiciones, la estructura ***for*** (para), es útil cuando se conoce el número de iteraciones proporcionadas por la variable de control del ciclo desde un valor inicial hasta un valor final con un paso que puede ser incremento o decremento. La estructura ***while*** (mientras), ejecuta el conjunto de sentencias delimitadas entre las llaves { y }, siempre que se cumpla la condición que controla el ciclo. La estructura ***do – while*** (haga – mientras), es similar a la estructura ***while*** salvo que la condición que controla el ciclo se evalúa al final del bloque, lo que implica que se ejecuta al menos una vez.

El capítulo 6 describe los ***arreglos unidimensionales*** o ***vectores***, es claro que en muchos programas se requiere almacenar un conjunto de datos para luego procesarlos e imprimir los resultados de los cálculos efectuados, se comienza con la definición de vector, la declaración de acuerdo con el tipo de dato, las diferentes formas de inicializar los elementos. El procesamiento de los elementos implica aplicar las estructuras repetitivas o iterativas, de las cuales se emplea la estructura ***for*** por su característica y facilidad para las iteraciones requeridas.

El manejo de ***tablas*** o ***matrices*** mediante ***arreglos bidimensionales*** se describe en el capítulo 7, se comienza con la definición del concepto de arreglo bidimensional, seguido de la forma de declarar las matrices según el tipo de dato, las diversas maneras de inicializar los elementos de la tabla, el procesamiento de los elementos de la matriz se ejecuta mediante estructuras repetitivas for para controlar las filas y para el control de las columnas de la tabla.

El capítulo 8, describe las ***funciones de usuario***, se inicia con la declaración de las variables globales y variables locales, para continuar con la definición de los conceptos de procedimiento y de funciones de usuario. Las funciones de usuario al igual que los procedimientos pueden requerir o no el paso de parámetros para su ejecución, por lo que se describen las dos modalidades básicas: paso de parámetros por valor y paso de parámetros por referencia. La diferencia sustancial entre los procedimientos y las funciones de usuario es que los primeros no retornan valor alguno, por lo cual los resultados de la ejecución de las diversas sentencias se realizan directamente en el procedimiento, en tanto que las funciones de usuario retornan valores de distinto tipo como: int (entero), float (de punto flotante), string (cadena) o double (real).

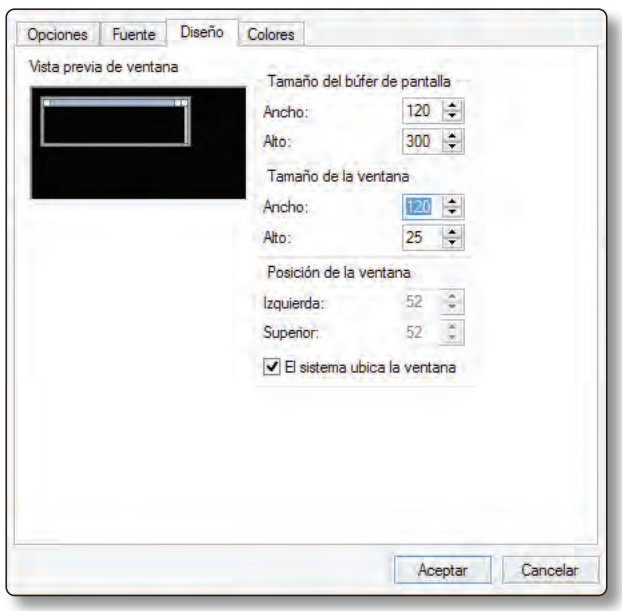
La ***recursividad*** se presenta en el capítulo 9, donde se describe el concepto, las características, los distintos tipos y las ventajas y desventajas de aplicar la recursividad en la solución de problemas relacionados.

En el capítulo 10 se ilustra el ***manejo de cadenas***, se define el concepto de cadena de caracteres, los tipos de datos string, la entrada y salida de cadenas de caracteres y las operaciones predefinidas.

Los ***registros o estructuras*** se ilustran en el capítulo 11, se describe el concepto de registro o estructura, las operaciones completas de asignación, paso como parámetros a procedimientos, estructuras como valor de retorno de funciones. Los miembros como estructuras y las operaciones correspondientes.

El capítulo 12 presenta el ***almacenamiento en memoria secundaria*** mediante el manejo de los ***ficheros o archivos***, se describe los archivos de texto y los binarios, los flujos de entrada y salida asociados a los archivos, la entrada de datos desde archivos de texto y la manipulación de los archivos (visualización, consulta, eliminación y adición de registros)

Por último, el capítulo 13 relacionado con la ***programación orientada a objetos POO***, describe el concepto de clases y sus características, los atributos y métodos, la instanciación, el tratamiento de las clases tanto en línea como a nivel modular, el manejo del concepto de herencia, de polimorfismo y de herencia múltiple.



Para obtener los resultados acorde con las imágenes indicadas en cada ejercicio, es conveniente ***configurar*** el ***tamaño*** de la ***ventana de salida***, para ello, se da clic en el ***icono*** de la ***parte superior izquierda*** de la ***ventana*** y luego en la opción ***propiedades*** de la ***ventana emergente***, se da clic en la pestaña ***Diseño*** y se ***configura*** el ***ancho y el alto de la ventana de salida***, lo recomendable para el ancho es 120 como se indica en la ***figura relacionada***.

1

GENERALIDADES DEL LENGUAJE DE PROGRAMACIÓN C++

1.1 CARACTERÍSTICAS

Creado a mediados de la década de los 80 por Bjarne Strousstrup, C++ es un lenguaje de programación considerado como una extensión del programa C, se caracteriza por abarcar tres aspectos de la programación[1]. (Olivares 2008)

1. Programación estructurada (paradigma de la programación basado en utilizar funciones o subrutinas y sólo tres estructuras de control: secuencia, selección o condicional e iteración o ciclo o bucle)[2]. (Covantec 2018).

2. Programación genérica (Se puede sintetizar en una palabra: **generalización**. Significa que, en la medida de lo posible, los algoritmos deben ser parametrizados al máximo y expresados de la forma más independiente posible de detalles concretos, permitiendo así que puedan servir para la mayor variedad posible de tipos y estructuras de datos)[3]. (Zator 2016)

3. Programación Orientada a Objetos (Se define como un paradigma de la programación, una manera de programar específica, donde se organiza el

1 *https://paginas.matem.unam.mx/pderbf/images/mprogintc++.pdf*

2 *https://entrenamiento-python-basico.readthedocs.io/es/latest/leccion5/programacion_estructurada.html*

3 *https://www.zator.com/Cpp/E4_12.htm*

código en unidades denominadas clases, de las cuales se crean objetos que se relacionan entre sí para conseguir los objetivos de las aplicaciones)[4]. (Desarrolloweb 2019).

1.2 ELEMENTOS BÁSICOS DE UN PROGRAMA

En todo programa se distinguen diferentes elementos del lenguaje C++, entre los que se destacan[5]. (Benjumea, V y Roldán, M 2017).

1.2.1 Instrucciones

Son cada una de las sentencias del programa que ejecutan diversas acciones, como la captura de datos, el procesamiento de datos y la salida de datos y resultados. Todas las instrucciones en C++ deben terminar con el símbolo punto y coma (;). Ejemplo de sentencias o instrucciones son:

```
system("cls");                              //Comando de limpieza de la pantalla
cout << "\n\tDígite el radio del círculo";  //Comando de salida (mensaje de texto)
cin >> ra;                                  //Comando de entrada (captura de
datos)
area = 3.1416 * pow(ra, 2);                 //Comando de procesamiento de datos
```

1.2.2 Datos y tipos

Los programas en C++ pueden procesar diferentes tipos de datos: numéricos, de carácter, booleanos y de cadena, tal como se describen en la tabla 1.

4 *https://desarrolloweb.com/articulos/499.php*

5 *http://www.lcc.uma.es/~vicente/docencia/cppdoc/programacion_cxx.pdf*

Tipo	Rango	Tamaño	Ejemplos
char	-128 a 127	1 Byte	$, ‘a’
string		28 Bytes	“Programa_C++”
bool	True, False	1 Byte	-12 > 12
short	-32768 a 32767	2 Bytes	-1545, 25749
int	-2147483648 a 2147483647	4 Bytes	-1048576, 1245689
long	-2147483648 a 2147483647	4 Bytes	-34502397, 123456
long long	-9223372036854775808 a 9223372036854775807	8 Bytes	-12547389545, 234589735712
unsigned char	0 a 255	1 Byte	227
unsigned short	0 a 65535	2 Bytes	35789
unsigned	0 a 4294967295	4 Bytes	315479234
unsigned long	0 a 4294967295	4 Bytes	40123579
unsigned long long	0 a 18446744073709551615	8 Bytes	112245467832
float	-1.17549435E-38 a 3.40282347E+38	4 Bytes	6.023E23
double	2.2250738585072014E-308 a 1.7976931348623157E+308	8 Bytes	3.141516, 2.7182
long double	3.36210314311209350626E-4932 a 1.18973149535723176502E+4932	12 Byte	-54628935E-150, 12357689342E1250

Tabla 1. Tipos de datos en C++. Fuente. Adaptado de[6] (Benjumea, V y Roldán, M 2017).

1.2.3 Bibliotecas

Una de las principales características del lenguaje de programación C++ es el amplio número de bibliotecas que provee y que se invocan cuando se requiera. En la estructura de un programa, las primeras líneas incluyen las bibliotecas necesarias para el correcto funcionamiento, por tal razón, se escriben tantas líneas ***#include*** como se requiera. Por ejemplo, la línea de código ***#include <iostream>,*** incluye la biblioteca ***iostream*** requerida cuando se establecen operaciones de ***entrada (lectura) o de salida (escritura) de datos***. Para emplear la biblioteca ***iostream*** es necesario utilizar el ***espacio de nombres std*** mediante la instrucción ***using namespace std;.***

6 *http://www.lcc.uma.es/~vicente/docencia/cppdoc/programacion_cxx.pdf*

1.2.4 Constantes simbólicas y constantes literales

El cálculo del área de un círculo se determina mediante la expresión $\pi * r^2$ donde π es el número pi cuyo valor es 3.1416 aproximadamente y r es el radio del círculo. Un fragmento de un programa en C++ que realiza el cálculo del área del círculo puede ser:

```
cout << "\n\tDígite el radio del círculo";
cin >> ra;
area = 3.1416 * pow(ra, 2);
```

Se observa que en el cálculo del área se asigna a la variable ***area*** el producto de multiplicar ***3.1416*** por el resultado de elevar el ***radio (ra) al cuadrado***. En dicho caso se emplea la ***constante 3.1416 de forma literal***.

Una variante del fragmento del programa anterior para el cálculo del área del círculo puede ser:

```
#define pi 3.1416
using namespace std;
int main()
{
   system("cls");            //Comando de limpieza de la pantalla
   system("color 70");       //Comando de configuración de color
   setlocale(LC_ALL, "");
   float ra, area, area1;
   cout << "\n\tLenguaje de programación C++";
   cout << "\n\tDígite el radio del círculo";
   cin >> ra;
   area = pi * pow(ra, 2);
   system("pause");             //Comando de pausa
}
```

En este caso el cálculo del área se realiza mediante la asignación a la variable ***area*** del producto de ***pi*** por el resultado de elevar el ***radio (ra) al cuadrado***, previamente se ha definido el nombre ***pi*** con el valor ***3.1416*** mediante la declaración ***#define pi 3.1416***. Por lo cual, se ha empleado una ***constante simbólica*** en el cálculo del área del círculo.

1.2.5 Identificadores

El identificador se define como el nombre que se asigna a cada elemento que se introduce en un programa en C++ que permite hacer referencia al mismo. El lenguaje C++ diferencia entre minúsculas y mayúsculas, en la siguiente sentencia se definen tres indicadores ra, area y area1 que corresponden a variables de tipo float.

```
float ra, area, area1;
```

1.2.6 Comentarios

En el lenguaje C++, los comentarios permiten hacer anotaciones o documentar partes o secciones de un programa, los comentarios pueden ser definidos en una línea empleando el símbolo `//` para indicar el comienzo o en un bloque para lo cual se requiere los símbolos `/*` al comienzo y `*/` al final del bloque. Las siguientes líneas incluyen comentarios de cada tipo.

```
// Programa que ordena los elementos de un vector de forma ascendente
/* Programa que dado un número ingresado por teclado, retorna su raíz cuadrada,
(emplee la función sqrt) y el cuadrado (emplee la función pow) */
```

1.2.7 Delimitadores

En un programa C++, es usual emplear símbolos como: ( ) { } , ; < > [ ] para especificar el comienzo o fin de una entidad, estos símbolos empleados se conocen como delimitadores. Por ejemplo, el símbolo ; se emplea para indicar el final de una instrucción o sentencia, los símbolos { y } se utilizan para delimitar el comienzo y fin de una función, procedimiento o de un bloque de sentencias.

1.2.8 Operadores

Son símbolos que se utilizan o emplean con significado propio para indicar un tipo de operación determinada. En la tabla 2 se describe el significado de los principales operadores matemáticos, relacionales, lógicos, de asignación e incrementales empleados en el lenguaje de programación C++.

Operadores matemáticos			
Símbolo	**Operación**	**Ejemplo**	**Resultado**
+	Suma	-8 + 23	15
-	Resta	32 - 23	9
*	Multiplicación	12 * 5	60
/	División	34 / 5	6.8
%	Módulo o residuo	34 % 5	4
Operadores relacionales			
<	Menor que	(-20 + 12) < 4 + 2 * 3	- 8 < 10
>	Mayor que	15 > 10 – 3 * (6 – 2 * 4)	15 > 16
<=	Menor o igual	3 / 4 <= 2 * (5 – 3 * 3) / 4	3 / 4 <= - 2
>=	Mayor o igual	25 >= - 2 * (7 – 3 * 5)	25 >= 16
==	Igual	2 + 5 * 4 == 66 – 4 * (6 + 5)	22 == 22
!=	Diferente	(2 – 4 * 6) != - 25 + 3	- 22 != - 22
Operadores lógicos			
&& and	Y lógica	(2 > 3 * 2) && (15 < 20)	False && True (False)
|| or	O lógico	(4 != 3 * 5) || (3 > 4 * 3)	True || False (True)
!	Negación lógica	! (12 – 3 * 2 == 5 * 3 - 9)	! True (False)
Operadores de asignación			
=	Igual	are = 2 * 3.1416 * 10	are = 62.832
+=	Más igual	a = 2; a += 5	a = 7
-=	Menos igual	a = 5; a -= 3	a = 2
*=	Por igual	a = 12; a *= 3	a = 36
/=	Dividido igual	a = 32; a /= 4	a = 8
Operadores incrementales y decrementales unitarios			
++	Incremento	x = 4; x++	x = 5
--	decremento	x = 4; x--	x = 3

Tabla 2. Tipos principales de operadores. Fuente. Elaboración propia.

1.2.9 Variables

En todo programa los datos iniciales y cálculos intermedios deben ser almacenados en memoria en variables de algún tipo, a diferencia de otros programas, en C++ las variables deben ser declaradas antes de utilizarse. Las variables se caracterizan por tener: un nombre que la identifica, un contenido, ser de algún tipo y la dirección en memoria.

Ejemplo 1

En la ejecución del siguiente fragmento de código, sí el usuario digita 10, los elementos de la variable ra son:

```
float ra;
cout << "\n\tDígite el radio del círculo";
cin >> ra;
cout << "\t" << &ra;
```

Nombre	ra
Contenido	10
Tipo de variable	float (real de punto flotante)
Dirección de memoria	001EFA98

Ejemplo 2

En la ejecución del siguiente fragmento de código, sí el usuario digita Fernando, los elementos de la variable nom son:

```
string nom;
cout << "\n\tDigite el nombre del usuario:\t";
cin >> nom;
cout << "\t" << &nom;
```

Nombre	nom
Contenido	Fernando
Tipo de variable	string (cadena)
Dirección de memoria	0108F960

Ejemplo 3

En la ejecución del siguiente fragmento de código, sí el usuario digita 25, los elementos de la variable edad son:

```
int edad;
cout << "\n\tDigite la edad del usuario:\t";
cin >> edad;
cout << "\t" << &edad;
```

Nombre	edad
Contenido	25
Tipo de variable	entero
Dirección de memoria	0108F954

Ejemplo 4

En la ejecución del siguiente fragmento de código, sí el usuario digita M, los elementos de la variable sex son:

```
char sex;
char* pchar;
pchar = &sex;
cout << "\n\tDigite el género del usuario:\t";
cin >> sex;
cout << "\n\tContenido " << sex;
cout << "\n\tMemoria:  " << &pchar;
```

Nombre	sex
Contenido	M
Tipo de variable	char (carácter)
Dirección de memoria	00D9F83C

1.2.10 Entrada / Salida

Por lo general un programa requerirá captar datos de entrada, procesarlos y mostrar los resultados. En el lenguaje C++ se emplea el flujo ***cin*** y el operador de extracción >> para la entrada de datos y ***cout*** con el operador de inserción << para la salida de datos. El siguiente fragmento de un programa ilustra los flujos delimitados.

```
cout << "\n\tDigite el nombre del usuario:\t";
cin >> nom;
cout << "\n\tEl nombre del usuario es:\t" << nom;
cout << "\n\tLa dirección de memoria de la variable nom es:\t" << &nom;
```

La primera línea muestra por pantalla el mensaje "Digite el nombre del usuario:", la segunda permite capturar lo que digita el usuario en la variable nom, las siguientes dos líneas muestran por pantalla el dato digitado por el usuario y la dirección de memoria de la variable nom.

1.2.11 Asignación

El operador de asignación = empleado en una instrucción o sentencia de asignación, permite adjudicar lo que se encuentra a la derecha del operador a la variable ubicada a la izquierda del mismo. A la derecha del operador puede ser un valor constante o el resultado de una expresión aritmética. En el siguiente fragmento de un programa se ilustra diferentes instrucciones de asignación.

```
float ang, anr, fusen, fucos, futan;
anr = ang * 3.1416 / 180;
fusen = sin(anr);
fucos = cos(anr),
futan = tan(anr);
```

En la primera línea se declaran las variables ang, anr, fusen, fucos, futan de tipo float, en la segunda línea se asigna el resultado de operar ang * 3.1416 / 180 a la variable anr, en la tercera línea se asigna el resultado de aplicar la función ***sin(anr)*** a la variable ***fusen***, en la siguiente de aplicar la función ***cos(anr)*** a la variable ***fucos*** y en la última de aplicar la función ***tan(anr)*** a la variable ***futan***.

1.2.12 Palabras reservadas

El lenguaje C++ emplea diversas palabras que tienen significado especial y que obviamente no pueden ser empleadas con otro sentido al crear un programa. Tales palabras hacen referencia a funciones propias del lenguaje, a estructuras o declaraciones de bibliotecas. Algunas de las palabras reservadas se ilustran en la tabla 3.

Palabras reservadas en el lenguaje C++				
main	*cin*	*cout*	*return*	*while*
for	*default*	*sin*	*cos*	*tan*
switch	*string*	*iostrem*	*double*	*float*
rand	*system*	*pause*	*pow*	*fixed*
setprecision	*case*	*break*	*class*	*struct*
private	*public*	*include*	*define*	*char*
int	*boolean*	*long*	*void*	*using*
color	*endl*	*time*	*null*	*and*
do	*namespace*	*sleep*	*srand*	*iomanip*

Tabla 3. Algunas palabras reservadas en el lenguaje C++.
Fuente. Adaptado de[7] (Benjumea, V y Roldán, M 2017).

7 *http://www.lcc.uma.es/~vicente/docencia/cppdoc/programacion_cxx.pdf*

1.2.13 Secuencias de escape

Son caracteres de control que normalmente se emplean para especificar ciertos caracteres especiales dentro de las cadenas de texto. En el lenguaje de programación C++ las secuencias de escape se resumen en la tabla 4, para su ejecución se requiere utilizar el carácter (\) barra invertida y por lo general una letra.

Secuencia de escape	Descripción	Representación
\’	Comilla simple	Byte 0x27
\”	Comilla doble	Byte 0x22
\?	Signo interrogación	Byte 0x3F
\\	Barra invertida	Byte 0x5C
\0	Carácter nulo	Byte 0x00
\a	Pitido	Byte 0x07
\b	Retorno	Byte 0x08
\f	Nueva página	Byte 0x0C
\n	Nueva línea	Byte 0x0A
\r	Retorno de carro	Byte 0x0D
\t	Tabulación horizontal	Byte 0x09
\v	Tabulación vertical	Byte 0x0B
\nnn	Valor en formato octal	Byte nnn
\Xnn	Valor en formato hexadecimal	Byte nn
\Unnnn	Valor en formato Unicode Puede dar como resultado varios caracteres	Código U+nnnn
\Unnnnnnnn	Valor en formato Unicode Puede dar como resultado varios caracteres	Código U+nnnnnnnn

Tabla 4. Secuencias de escape o caracteres de control. Fuente. Obtenido de[8] (cppreference.com 2018).

1.2.14 Tabla ASCII

ASCII es un acrónimo de la expresión inglesa ***American Standard Code for Information Interchange*** que traducido equivale a ***Código Estándar para el Intercambio de Información***. Corresponde a un patrón de codificación numérico empleado para representar 128 caracteres diferentes utilizando una escala decimal del 0 al 127, se requiere para ello **7 bits** (2^7 = 128 permutaciones). Generalmente el código ***ASCII*** se amplifica a **8 bits** (**1 byte**) añadiendo un bit de control, llamado bit de paridad.

8 *https://es.cppreference.com/w/cpp/language/escape*

En la tabla 5, se describe cada uno de los 128 códigos ***ASCII*** codificados en los sistemas numéricos: ***decimal*** (base 10), ***octal*** (base 8), ***hexadecimal*** (base 16) y el carácter (CH) que representa.

dec	oct	hex	ch	dec	oct	hex	ch	dec	oct	hex	ch	dec	oct	hex	ch
0	0	00	NUL (null)	32	40	20	(space)	64	100	40	@	96	140	60	`
1	1	01	SOH (start of header)	33	41	21	!	65	101	41	A	97	141	61	a
2	2	02	STX (start of text)	34	42	22	“	66	102	42	B	98	142	62	b
3	3	03	ETX (end of text)	35	43	23	#	67	103	43	C	99	143	63	c
4	4	04	EOT (end of transmission)	36	44	24	$	68	104	44	D	100	144	64	d
5	5	05	ENQ (enquiry)	37	45	25	%	69	105	45	E	101	145	65	e
6	6	06	ACK (acknowledge)	38	46	26	&	70	106	46	F	102	146	66	f
7	7	07	BEL (bell)	39	47	27	‘	71	107	47	G	103	147	67	g
8	10	08	BS (backspace)	40	50	28	(	72	110	48	H	104	150	68	h
9	11	09	HT (horizontal tab)	41	51	29	)	73	111	49	I	105	151	69	i
10	12	0a	LF (line feed - new line)	42	52	2a	*	74	112	4a	J	106	152	6a	j
11	13	0b	VT (vertical tab)	43	53	2b	+	75	113	4b	K	107	153	6b	k
12	14	0c	FF (form feed - new page)	44	54	2c	,	76	114	4c	L	108	154	6c	l
13	15	0d	CR (carriage return)	45	55	2d	-	77	115	4d	M	109	155	6d	m
14	16	0e	SO (shift out)	46	56	2e	.	78	116	4e	N	110	156	6e	n
15	17	0f	SI (shift in)	47	57	2f	/	79	117	4f	O	111	157	6f	o
16	20	10	DLE (data link escape)	48	60	30	0	80	120	50	P	112	160	70	p
17	21	11	DC1 (device control 1)	49	61	31	1	81	121	51	Q	113	161	71	q
18	22	12	DC2 (device control 2)	50	62	32	2	82	122	52	R	114	162	72	r
19	23	13	DC3 (device control 3)	51	63	33	3	83	123	53	S	115	163	73	s
20	24	14	DC4 (device control 4)	52	64	34	4	84	124	54	T	116	164	74	t
21	25	15	NAK (negative acknowledge)	53	65	35	5	85	125	55	U	117	165	75	u
22	26	16	SYN (synchronous idle)	54	66	36	6	86	126	56	V	118	166	76	v
23	27	17	ETB (end of transmission block)	55	67	37	7	87	127	57	W	119	167	77	w
24	30	18	CAN (cancel)	56	70	38	8	88	130	58	X	120	170	78	x
25	31	19	EM (end of medium)	57	71	39	9	89	131	59	Y	121	171	79	y
26	32	1a	SUB (substitute)	58	72	3a	:	90	132	5a	Z	122	172	7a	z
27	33	1b	ESC (escape)	59	73	3b	;	91	133	5b	[	123	173	7b	{
28	34	1c	FS (file separator)	60	74	3c	<	92	134	5c	\	124	174	7c	|
29	35	1d	GS (group separator)	61	75	3d	=	93	135	5d	]	125	175	7d	}
30	36	1e	RS (record separator)	62	76	3e	>	94	136	5e	^	126	176	7e	~
31	37	1f	US (unit separator)	63	77	3f	?	95	137	5f	_	127	177	7f	DEL (delete)

Tabla 5. Tabla ASCII. Fuente. Obtenido de[9] (cppreference.com 2018).

9 *https://es.cppreference.com/w/cpp/language/escape*

1.3 FASES DE LA EJECUCIÓN DE UN PROGRAMA

La conversión de un algoritmo en un programa ejecutable requiere en general de distintas fases, en la tabla 5 se describen algunas de las principales características de las fases.

Fase	Descripción	Archivo de salida	Archivo incorporado
Edición	Se convierte el algoritmo en instrucciones del lenguaje C++ empleando un editor de texto para obtener el código fuente	Archivo de extensión .cpp (código fuente)	
Preproceso	Facilita procesar el código fuente e incluir los archivos de cabecera que provean al compilador la información requerida de las funciones de biblioteca		Archivos de cabecera extensión .h
Compilación	Traduce el código fuente modificado a código de máquina o código objeto	Archivo de extensión .obj (código objeto)	
Enlace	Requiere de un programa llamado enlazador o linkador que une el código objeto de las funciones de biblioteca utilizadas con el código objeto del programa que las utiliza	Archivo de extensión .exe (Archivo ejecutable)	Bibliotecas empleadas
Ejecución	Mediante el archivo .exe generado se pone en marcha el programa		

Tabla 6. Fases en la ejecución de un programa en C++. Fuente. Adaptado de[10] (Carlospes s.f).

1.4 TIPOS DE ERRORES EN LA EJECUCIÓN DE UN PROGRAMA

Es posible que cuando se compile un programa, aparezcan errores que deben ser identificados y corregidos para la correcta ejecución del programa, la ***depuración*** es el proceso mediante el cual se detectan y corrigen los errores resultantes de la fase de compilación en la ejecución de un programa. Los errores pueden ser de diferentes tipos.

1.4.1 Errores de tiempo de compilación

Son errores detectados por el preprocesador, el analizador sintáctico y el propio compilador. Los comunes son errores de sintaxis como se observa en el siguiente fragmento de código.

10 *http://www.carlospes.com/curso_de_lenguaje_c/01_02_fases.php*

```
int edad;
cout << "\n\tDigite la edad del usuario:\t"
cin >> edades;
```

En la segunda línea se presenta el error de falta del carácter ***; (punto y coma)*** que indica el final de la sentencia o instrucción, en la tercera línea el error obedece a que se intenta leer en la variable edades que no está declarada, ya que la variable declarada es edad.

1.4.2 Errores de tiempo de enlazado

Son detectados por el enlazador, ejemplo de este tipo de errores es el llamado a una función cuya definición o estructura no aparece por ningún lado, como se observa en el siguiente fragmento de código.

```
#include <iostream>
#include <string>
#include <windows.h>
#include <time.h>
#include <locale.h>
using namespace std;
void captura();
void procesa();
int main()
{
   system("cls");                  //Comando de limpieza de la pantalla
   system("color 70");             //Comando de configuración de color
   setlocale(LC_ALL, "");
   string nom;
   int edad;
   char sex;
   cout << endl;
   cout << "\t";
   cout << "\n\tLenguaje de programación C++";
   captura();
   system("pause");
   return 0;
}
```

En el fragmento de código se observa que se declara la función ***captura()*** de tipo ***vacío (void)*** que no retorna valor alguno ni requiere parámetros, luego se hace el llamado a la función pero falta la definición de la estructura de la misma. De igual forma se declara la función ***procesa()*** que no retorna valor alguno porque es de tipo ***void***, ni requiere parámetros para su ejecución, pero no se realiza el llamado ni está definida su estructura.

1.4.3 Errores de tiempo de ejecución (runtime)

Son aquellos errores que se producen cuando se ejecuta o corre el programa, son más difíciles de diagnosticar en especial en programas o aplicaciones relativamente grandes. Por ejemplo error de tiempo de ejecución es "un servicio de red no disponible" o "memoria insuficiente".

1.5 EJEMPLO DE UN PROGRAMA

El código de programa que se presenta en la tabla 6, permite al usuario ingresar un número para adivinar un número generado de forma aleatoria simulando el lanzamiento de un dado numerado del uno (1) al seis (6). Los elementos del programa incluyen:

- *Bloque de comentarios*. Las primeras cinco líneas delimitadas por los caracteres /* que determinan el inicio del bloque y */ para indicar el fin del bloque.
- *Directivas del preprocesador*. Líneas de la seis a la diez en la que se definen las bibliotecas utilizadas ***iostream*** (uso del flujo de entrada y de salida), ***string*** (definir variables tipo cadena), ***locale.h*** (uso de caracteres especiales como acentos) y ***time.h*** (biblioteca que contiene funciones para manipular y formatear la fecha y hora del sistema).
- *Prototipo función de usuario*. Se declara la función de usuario ***procesar()*** que no retorna valor ya que es de tipo ***vacío (void)*** y tampoco requiere parámetros para su ejecución.
- *Función principal*. Función ***main*** de tipo entero cuyo bloque de instrucciones está delimitado por los caracteres **{** de inicio y **}** de finalización, contiene un bucle determinado por la sentencia repetitiva **while** que le brinda al usuario la oportunidad de volver a procesar o jugar, además de la llamada a la función de usuario ***procesar***.

- *Estructura de la función de usuario procesar*. La función de usuario procesar es de tipo ***vacío (void)*** por lo que no retorna valor ni requiere parámetros para su ejecución, la función captura el valor ingresado por el usuario validándolo en el rango de 1 a 6, lo compara con el número generado de forma aleatoria y determina sí es igual, en cuyo caso muestra el mensaje "Felicidades, adivinó el número" y muestra el número, en caso contrario le indica al usuario que "No adivinó el número".

Línea	Instrucción / Sentencia	Descripción
1	`/* Programa que simula el juego de adivinar un número en el`	Bloque de comentarios delimitados por los caracteres /* inicio y */ final
2	`lanzamiento de un dado numerado del 1 al 6`	
3	`Autores:  JFBU / IYPG`	
4	`Año:  2020`	
5	`*/`	
6	`#include <iostream>`	Directivas del preprocesador (Bibliotecas y nombre de espacio estándar)
7	`#include <string>`	
8	`#include <locale.h>`	
9	`#include <time.h>`	
10	`using namespace std;`	
11	`void procesar();`	Prototipo función
12	`int main()`	Función principal
13	`{`	
14	`    setlocale(LC_ALL, "");`	
15	`    string seguir = "S";`	
16	`    srand(time(NULL));`	
17	`    while (seguir == "S" or seguir == "s")`	
18	`    {`	
19	`        system("cls");`	
20	`        system("color 70");`	
21	`        procesar();`	Llamada función
22	`        cout << "\n\tDesea continuar jugando S o N\t";`	
23	`        cin >> seguir;`	
24	`    }`	
25	`    cout << endl << endl;`	
26	`    system("pause");`	
27	`    return 0;`	
28	`}`	

Estructura de la función de usuario procesar

```
29      void procesar()
30      {
31          int nc, nu;
32          nc = rand() % 6 + 1;
33          cout << “\n\tDigite el número entre 1 y 6\t”;
34          cin >> nu;
35          while (nu < 1 or nu > 6)
36          {
37              cout << “\n\tDato errado, digite el número entre 1 y
        6\t”;
38              cin >> nu;
39          }
40          if (nc == nu)
41          {
42              cout << “\n\tFelicidades, adivinó el número:\t” <<
        nc << endl;
43          }
44          else
45              cout << “\n\tNo adivinó el número:\t” << endl;
46      }
```

Tabla 7. Ejemplo de un programa en C++. Fuente. Elaboración propia.

2

ESTRUCTURAS SECUENCIALES

2.1 SENTENCIA

La ***sentencia*** corresponde a la unidad elemental de acción en un programa en el lenguaje C++ que termina con el carácter punto y coma (;). Las sentencias pueden ser de distinta índole: sentencias de captura de datos mediante el flujo ***cin*** y el operador de extracción >>, de salida de datos mediante el flujo ***cout*** con el operador de inserción <<, de asignación empleando el operador =.

2.2 ESTRUCTURA SECUENCIAL

Una ***estructura secuencial*** es el conjunto de sentencias que se ejecutan una a continuación de otra y que se encuentran dentro de un bloque delimitado por los caracteres llave de inicio { y llave de finalización }. El siguiente programa de ejemplo ilustra distintos tipos de sentencias en una estructura secuencial para el cálculo de la densidad de una sustancia a partir de la captura de la masa y del volumen.

```
//Ejemplo programa estructura secuencial
//Cálculo de la densidad de un  cuerpo
#include <iostream>
#include <string>
#include <locale.h>
#include <windows.h>
using namespace std;
int main()
{
    setlocale(LC_ALL, “”);                    //Sentencia función localización
    float masa, volu, densi;                  //Sentencia definición variables
```

```
    system("cls");                                  //Sentencia limpieza de pantalla
    system("color 70");                             //Sentencia definición de color
    cout << "\n\tDigite masa en g:\t";              //Sentencia mensaje de salida
    cin >> masa;                                    //Sentencia entrada de datos
    cout << "\n\tDigite volumen en ml:\t";          //Sentencia mensaje de salida
    cin >> volu;                                    //Sentencia entrada de datos
    densi = (float(masa)) / volu;                   //Sentencia de asignación
    cout << "\n\tMasa en g:\t" << masa;             //Sentencia salida de datos
    Sleep(500);                                     //Sentencia retardo de tiempo
    cout << "\n\tVolumen en ml:\t" << volu;         //Sentencia salida de datos
    Sleep(500);                                     //Sentencia retardo de tiempo
    cout << "\n\tDensidad en g/ml:\t" << densi;     //Sentencia salida de resultado
    Sleep(500);                                     //Sentencia retardo de tiempo
    cout << endl << endl;                           //Sentencia ruptura de línea
    system("pause");                                //Sentencia pausa de sistema
    return 0;                                       //Sentencia retorno de función
}
```

2.3 EJERCICIOS RESUELTOS DE ESTRUCTURAS SECUENCIALES

Ejercicio 1

Escriba, compile y ejecute un programa en C++, que dado un número ingresado por teclado retorne su raíz cuadrada (emplee la función sqrt) y el cuadrado (emplee la función pow). Se debe mostrar el número y los resultados de los cálculos efectuados.

```
/* Escriba, compile y ejecute un programa en C++, que dado un número ingresado por
teclado retorne su raíz cuadrada y el cuadrado. */
#include <iostream>
#include <string>
#include <windows.h>
#include <locale.h>
using namespace std;
int main()
{
   float num, raiz, cua;      //Definición de variables reales
   string seguir = "S";       //Definición de variable tipo cadena y asignación
   setlocale(LC_ALL, "");
   while (seguir == "S" or seguir == "s")
   {
      system("cls");          //Comando para limpiar la pantalla
      cout << "\n\tPrograma que calcula la raíz y el cuadrado de un número!";
      cout << "\n\t-------------------------------------------------------\n\n";
      cout << "\n\tDigite un número:\t";
```

```
        cin >> num;             //Captura el número digitado
        system("color 70");     //Configura el color
        if (num < 0)            //Valida si el número es negativo
           cout << "\n\tError: El número no tiene raíz real\n";
        else
           raiz = sqrt(num);    //Calcula la raíz cuadrada del número
        cua = pow(num, 2);      //Calcula el cuadrado del número
        cout << "\n\n\t         Imprimir datos y resultados";
        cout << "\n\t-------------------------------------------\n\n";
        cout << "\n\t El número digitado es:\t\t" << num << endl << endl;
        if (num >= 0)
           cout << "\n\t La raíz cuadrada es: \t\t" << "+/- " << raiz << "\n\n";
        cout << "\n\t El cuadrado del número es:\t" << cua << "\n\n";
        cout << "\n\t-------------------------------------------\n\n";
        cout << "\n\tDesea continuar procesando S o N\t";
        cin >> seguir;          //Variable de control del ciclo
        while (seguir != "S" and seguir != "s" and seguir != "N" and seguir != "n")
        {
           cout << "\n\tError. Digite S o  N\t";
           Sleep(500); cin >> seguir;
        }
     }
     cout << endl << endl;
     system("pause");
     return 0;
}
```

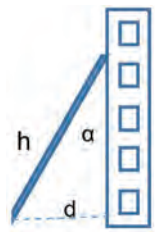
El resultado de la ejecución del programa se ilustra en la figura 1.

```
Programa que calcula la raíz y el cuadrado de un número!
--------------------------------------------------------

Digite un número:       25

        Imprimir datos y resultados
-------------------------------------------

 El número digitado es:         25

 La raíz cuadrada es:           +/- 5

 El cuadrado del número es:     625

-------------------------------------------

Desea continuar procesando S o N
```

Figura 1. Resultado de la ejecución del programa que obtiene la raíz cuadrada y el cuadrado de un número. Fuente. Obtenido mediante (Visual Studio 2019).

Ejercicio 2

Teniendo en cuenta la figura relacionada, escriba, compile y ejecute un programa en C++ que determine la distancia de la base de la escalera al edificio, se debe mostrar los datos iniciales que se leen por teclado (hipotenusa en metros y el ángulo en grados) y el resultado del cálculo efectuado.




```
/* Escriba, compile y ejecute un programa en C++ que determine la distancia de una
escalera apoyada a la base de un edificio. */
#include <iostream>
#include <string>
#include <locale.h>
#include <windows.h>
using namespace std;
int main()
{
   float dis, ang, hipot;
   string seguir = "S";
   setlocale(LC_ALL, "");
   while (seguir == "S" or seguir == "s")
   {
      system("cls");            //Comando de limpieza de la pantalla
      system("color 70");       //Comando de configuración de color
      cout << "\n\n\n\tPrograma que calcula distancia de la base de la escalera al
edificio!\n";
      cout << "\t-----------------------------------------------------------------
---\n";
      cout << "\n\n\tDigite los datos de la hipotenusa en m y el ángulo en grados >
0 y < 90\n\n\t";
      cin >> hipot >> ang;      //Lectura de los datos de entrada
      while (hipot < 0 or ang <= 0 or ang >= 90)
      {
         cout << "\n\tError, digite datos positivos de la hipotenusa y el ángulo >
0 y < 90\n\n\t";
         cin >> hipot >> ang;   //Lectura de los datos de entrada
      }
      dis = ((float)hipot) * sin(ang * 3.1416 / 180);     //Cálculo de la distancia
```

```
        cout << "\n\t              Imprimir datos y resultados  \n";
        cout << "\t-------------------------------------------------------\n";
        cout << endl;
        cout << "\n\thipotenusa (longitud de la escalera en m): \t" << hipot << endl
<< endl;
        cout << "\n\tÁngulo en grados de la escalera:           \t" << ang << endl <<
endl;
        cout << "\n\tDistancia de la escalera al edificio en m: \t" << dis << endl <<
endl;
        cout << "\n\t-------------------------------------------------------\n";
        cout << "\n\tDesea continuar procesando S o N\t";
        cin >> seguir;
        while (seguir != "S" and seguir != "s" and seguir != "N" and seguir != "n")
        {
           cout << "\n\tError. Digite S o  N\t";
           Sleep(500);
           cin >> seguir;
        }
    }
    cout << endl << endl;
    cout << "\t";
    system("pause");
    return 0;
}
```

El resultado de la ejecución del programa se ilustra en la figura 2.

```
Programa que calcula distancia de la base de la escalera al edificio!
-----------------------------------------------------------------------

Digite los datos de la hipotenusa en m y el ángulo en grados > 0 y < 90
20      60
              Imprimir datos y resultados
-------------------------------------------------------

hipotenusa (longitud de la escalera en m):      20

ángulo en grados de la escalera:                60

Distancia de la escalera al edificio en m:      17.3205

-------------------------------------------------------

Desea continuar procesando S o N
```

Figura 2. Resultado de la ejecución del programa que calcula la distancia de la base de la escalera al edificio. Fuente. Obtenido mediante (Visual Studio 2019).

Ejercicio 3

Escriba, compile y ejecute un programa en C++ que convierta la temperatura en grados Celsius a grados Fahrenheit, Kelvin y Rankime, se debe leer la temperatura en grados Celsius y validar que no puede ser menor de -273. Aplique las siguientes fórmulas: ***°K = °C + 273, °F = 9/5 * °C + 32 y °R = °F + 460***. Se debe imprimir la temperatura en cada escala.

```
/* Escriba, compile y ejecute un programa en C++ que convierta la temperatura en
grados Celsius a grados Fahrenheit, Kelvin y Rankime. */
#include <iostream>
#include <string>
#include <locale.h>
#include <windows.h>
using namespace std;
int main()
{
   float C, K, F, R;
   string seguir = "S";
   while (seguir == "S" or seguir == "s")
   {
      system("cls");         //Comando de limpieza de la pantalla
      system("color 70");    //Comando de configuración de color
      setlocale(LC_ALL, "");
      cout << endl;
      cout << "\n\n\tPrograma que convierte de una escala de temperatura a
otra!\n";
      cout << "\t-------------------------------------------------------------\n";
      cout << "\n\tDigite la temperatura en grados celsius\n\n\t";
      cin >> C;              //Lectura de los datos de entrada
      while (C < -273)
      {
         cout << "\n\tDato errado, digite la temperatura en grados celsius de
nuevo\t";
         cin >> C;
      }
      K = C + 273; F = 9.0 / 5 * C + 32; R = F + 460;    //Conversión de
temperaturas
      cout << "\n\t           Imprimir datos y resultados  \n";
      cout << "\t-----------------------------------------------------\n";
      cout << endl;
      cout << "\n\tLa temperatura en grados Celsius es:    \t" << C << endl;
      cout << "\n\tLa temperatura en grados Kelvin es:     \t" << K << endl;
      cout << "\n\tLa temperatura en grados Fahrenheit es: \t" << F << endl;
```

```
        cout << "\n\tLa temperatura en grados Rankime es:    \t" << R << endl;
        cout << "\n\t-----------------------------------------------------\n";
        cout << "\n\tDesea continuar procesando S o N\t";
        cin >> seguir;
        while (seguir != "S" and seguir != "s" and seguir != "N" and seguir != "n")
        {
           cout << "\n\tError. Digite S o  N\t";
           Sleep(500);
           cin >> seguir;
        }
    }
    cout << endl << endl;
    cout << "\t";
    system("pause");
    return 0;
}
```

El resultado de la ejecución del programa se ilustra en la figura 3.

```
Programa que convierte de una escala de temperatura a otra!
-------------------------------------------------------------

Digite la temperatura en grados celsius
25
            Imprimir datos y resultados
-------------------------------------------------------

La temperatura en grados Celsius es:            25

La temperatura en grados Kelvin es:             298

La temperatura en grados Fahrenheit es:         77

La temperatura en grados Rankime es:            537

-------------------------------------------------------

Desea continuar procesando S o N
```

Figura 3. Resultado de la ejecución del programa que convierte de una escala de temperatura a otra. Fuente. Obtenido mediante (Visual Studio 2019).

Ejercicio 4

Escriba, compile y ejecute un programa en C++ que calcule el volumen de titulación requerido de un ácido cuya concentración se lee por teclado respecto al volumen y concentración de una base cuyos datos se leen por teclado. Aplique la expresión ***Na* * *Va* = *Nb* * *Vb*** donde ***Na*** es la concentración normal del ácido en eq-g/L, ***Va*** es el volumen del ácido en litros L, ***Nb*** es la concentración normal de la base en eq-g/L y ***Vb*** es el volumen de la base en litros.

```
/* Escriba, compile y ejecute un programa en C++ que calcule el volumen de
titulación requerido de un ácido. */
#include <iostream>
#include <string>
#include <locale.h>
#include <windows.h>
using namespace std;
int main()
{
   float na, va, nb, vb;
   string seguir = "S";
   setlocale(LC_ALL, "");
   while (seguir == "S" or seguir == "s")
   {
      system("cls");                  //Comando de limpieza de la pantalla
      system("color 70");             //Comando de configuración de color
      cout << "\n\n\tPrograma que calcula el volumen de titulación de una base!\n";
      cout << "\t-------------------------------------------------------------\n";
      cout << "\n\tDigite la concentración en eq-g/L y volumen de la base en
L:\n\t";
      cin >> nb >> vb;                //Lectura de los datos de entrada
      while (nb <= 0 or vb <= 0)      //Validación de datos
      {
         cout << "\n\tError. Digite la concentración en eq-g/L y volumen de la base
en L:\n\t";
         cin >> nb >> vb;             //Lectura de los datos de entrada
      }
      cout << "\n\tDigite la concentración del ácido en eq-g/L\t";
      cin >> na;
      while (na <= 0)                 //Validación de datos
      {
         cout << "\n\tError. Digite la concentración del ácido en eq-g/L\t";
         cin >> na;
      }
      va = ((float)nb) * vb / na;     //Cálculo del volumen de titulación
```

```
        cout << "\n\n\t                Imprimir datos y resultados  \n";
        cout << "\t-------------------------------------------------------------\n";
        cout << endl;
        cout << "\n\tLa concentración de la base en eq-g/L es:      \t" << nb << endl;
        cout << "\n\tEl volumen de la base en litros L es:          \t" << vb << endl;
        cout << "\n\tLa concentración del ácido en eq-g/L es:       \t" << na << endl;
        cout << "\n\tEl volumen requerido del ácido en litros L es: \t" << va << endl;
        cout << "\n\t-------------------------------------------------------------\n";
        cout << "\n\tDesea continuar procesando S o N\t";
        cin >> seguir;
        while (seguir != "S" and seguir != "s" and seguir != "N" and seguir != "n")
        {
           cout << "\n\tError. Digite S o  N\t";
           Sleep(500);
           cin >> seguir;
        }
    }
    cout << endl << endl;
    cout << "\t";
    system("pause");
    return 0;
}
```

El resultado de la ejecución del programa se ilustra en la figura 4.

```
Programa que calcula el volumen de titulación de una base!
----------------------------------------------------------

Digite la concentración en eq-g/L y volumen de la base en L:
0.5     1.8

Digite la concentración del ácido en eq-g/L     1.5


                Imprimir datos y resultados
-------------------------------------------------------------


La concentración de la base en eq-g/L es:       0.5

El volumen de la base en litros L es:           1.8

La concentración del ácido en eq-g/L es:        1.5

El volumen requerido del ácido en litros L es:  0.6

-------------------------------------------------------------

Desea continuar procesando S o N
```

Figura 4. Resultado de la ejecución del programa que calcula el volumen de titulación de una base. Fuente. Obtenido mediante (Visual Studio 2019).

Ejercicio 5

Escriba, compile y ejecute un programa en C++ que calcule las raíces de una ecuación cuadrática de la forma $a X^2 + b X + c$, donde por teclado se leen los coeficientes ***a, b y c***. Se debe mostrar la ecuación cuadrática y las raíces.

```
/* Escriba, compile y ejecute un programa en C++ que calcule las raíces de una
ecuación cuadrática de la forma aX2 + bX + c. */
#include <iostream>
#include <string>
#include <locale.h>
#include <windows.h>
using namespace std;
int main()
{
   float a, b, c, r1, r2;
   string seguir = "S";
   setlocale(LC_ALL, "");
   while (seguir == "S" or seguir == "s")
   {
      system("cls");            //Comando de limpieza de la pantalla
      system("color 70");       //Comando de configuración de color
      cout << endl;
      cout << "\n\tPrograma que calcula las raíces de una ecuación cuadrática!\n";
      cout << "\t-------------------------------------------------------------\n";
      cout << "\n\tDigite los coeficientes a, b y c de la ecuación cuadrática:\n\t";
      cin >> a >> b >> c;       //Lectura de los datos de entrada
      cout << endl;
      while (a == 0)            //Validación de datos
      {
         cout << "\n\tNo es una ecuación cuadrática, digite el coeficiente a\t";
         cin >> a;              //Lectura de los datos de entrada
      }
      if (b * b - 4 * a * c < 0)
      {
         cout << "\n\tLa ecuación no tiene raíces reales\n";
         system("pause");
         return 0;
      }
      r1 = (-b + sqrt(b * b - 4 * a * c)) / (2.0 * a);  //Cálculo de la primera raíz
      r2 = (-b - sqrt(b * b - 4 * a * c)) / (2.0 * a);  //Cálculo de la segunda raíz
      cout << "\n\t             Imprimir datos y resultados  \n";
      cout << "\t-------------------------------------------------------\n";
      cout << "\n\tEcuación cuadrática general   a X ^ 2  +  b X  +  c   \n\n";
```

```
      cout << "\n\tEcuación cuadrática particular " << a << " X ^ 2  +  " << b << "
X  +  " << c << "\n\n";
      cout << "\n\tEl coeficiente cuadrático es:                \t" << a << endl;
      cout << "\n\tEl coeficiente lineal es:                    \t" << b << endl;
      cout << "\n\tEl coeficiente termino independiente es:     \t" << c << endl;
      cout << "\n\tLa primera raíz de la ecuación cuadrática es:\t" << r1 << endl;
      cout << "\n\tLa segunda raíz de la ecuación cuadrática es:\t" << r2 << endl;
      cout << "\n\t-------------------------------------------------------\n";
      cout << "\n\tDesea continuar procesando S o N\t"; cin >> seguir;
      while (seguir != "S" and seguir != "s" and seguir != "N" and seguir != "n")
      {
         cout << "\n\tError. Digite S o  N\t";
         Sleep(500); cin >> seguir;
      }
   }
   cout << endl << endl;
   cout << "\t";
   system("pause");
   return 0;
}
```

El resultado de la ejecución del programa se ilustra en la figura 5.

```
Programa que calcula las raíces de una ecuación cuadrática!
-------------------------------------------------------

Digite los coeficientes a, b y c de la ecuación cuadrática:
3    15    18

              Imprimir datos y resultados
-------------------------------------------------------

Ecuación cuadrática general   a X ^ 2  +  b X  +  c

Ecuación cuadrática particular 3 X ^ 2  +  15 X  +  18

El coeficiente cuadrático es:                    3
El coeficiente lineal es:                        15
El coeficiente termino independiente es:         18
La primera raíz de la ecuación cuadrática es:    -2
La segunda raíz de la ecuación cuadrática es:    -3
-------------------------------------------------------

Desea continuar procesando S o N
```

Figura 5. Resultado de la ejecución del programa que calcula las raíces de la ecuación cuadrática. Fuente. Obtenido mediante (Visual Studio 2019).

Ejercicio 6

Escriba, compile y ejecute un programa en C++ que calcule la nota final del estudiante conocidos tres notas parciales que se leen por teclado al igual que el nombre del estudiante, las notas parciales tienen ponderaciones de 0.25, 0.30 y 0.45 respectivamente. Se debe mostrar el nombre del estudiante, las notas parciales, las ponderaciones y la nota final.

```
/* Escriba, compile y ejecute un programa en C++ que calcule la nota final del
estudiante conocidos tres notas parciales que se leen por teclado. */
#include <iostream>
#include <string>
#include <locale.h>
#include <windows.h>
using namespace std;
int main()
{
   float n1, n2, n3, nf;
   string seguir = "S", nom;
   setlocale(LC_ALL, "");
   while (seguir == "S" or seguir == "s")
   {
      system("cls");                        //Comando de limpieza de la pantalla
      system("color 70");                   //Comando de configuración de color
      cout << "\n\n\tPrograma que calcula la nota final del estudiante!\n";
      cout << "\t--------------------------------------------------\n";
      cout << "\n\n\tDigite el apellido del estudiante\t";
      cin >> nom;                           //Lectura de los datos de entrada
      cout << "\n\n\tDigite las tres notas parciales entre 0 y 5\n\t";
      cin >> n1 >> n2 >> n3;
      while (n1 < 0 or n1 > 5 or n2 < 0 or n2 > 5 or n3 < 0 or n3 > 5)//Valida
datos
      {
         cout << "\n\n\tError, digite las tres notas parciales entre 0 y 5\n\t";
         cin >> n1 >> n2 >> n3;             //Lectura de los datos de entrada
      }
      nf = n1 * 0.25 + n2 * 0.3 + n3 * 0.45; //Calculo nota final
      cout << "\n\n\n\t                  Imprimir datos y resultados  \n";
      cout << "\t-------------------------------------------------------------
\n";
      cout << "\n\tReporte de calificaciones del estudiante:      \t" << nom <<
endl;
      cout << "\n\tLa primera nota parcial y su porcentaje son:  \t" << n1 << "\
t25%" << endl;
```

```
      cout << "\n\tLa segunda nota parcial y su porcentaje son:  \t" << n2 << "\
t30%" << endl;
      cout << "\n\tLa tercera nota parcial y su porcentaje son:  \t" << n3 << "\
t45%" << endl;
      cout << "\n\tLa nota final del estudiante es:              \t" << nf << endl;
      cout << "\n\t-------------------------------------------------------------
\n";
      cout << "\n\tDesea procesar otro estudiante S o N\t"; cin >> seguir;
      while (seguir != "S" and seguir != "s" and seguir != "N" and seguir != "n")
      {
         cout << "\n\tError. Digite S o  N\t";
         Sleep(500);
         cin >> seguir;
      }
   }
   cout << "\n\n\t";
   system("pause");
   return 0;
}
```

El resultado de la ejecución del programa se ilustra en la figura 6.

```
Programa que calcula la nota final del estudiante!
--------------------------------------------------


Digite el apellido del estudiante       Castillo


Digite las tres notas parciales entre 0 y 5
3.4     2.5     4.2


                    Imprimir datos y resultados
-----------------------------------------------------------------

Reporte de calificaciones del estudiante:       Castillo

La primera nota parcial y su porcentaje son:    3.4     25%

La segunda nota parcial y su porcentaje son:    2.5     30%

La tercera nota parcial y su porcentaje son:    4.2     45%

La nota final del estudiante es:                3.49

-----------------------------------------------------------------

Desea procesar otro estudiante S o N
```

Figura 6. Resultado de la ejecución del programa que calcula la nota final del estudiante. Fuente. Obtenido mediante (Visual Studio 2019).

Ejercicio 7

Escriba, compile y ejecute un programa en C++ que calcule la altura máxima y el alcance de un proyectil que es disparado con un ***ángulo α*** y con una ***velocidad*** v_i describiendo un movimiento parabólico. Emplee las siguientes fórmulas del movimiento parabólico: $x\ máx = v_i^2 * sen\ (2\alpha) / g$ y $h\ máx = v_i^2 * sen^2\ (\alpha) / 2g$, ***donde*** v_i es la velocidad inicial, ***α*** es el ángulo de salida y ***g*** es la aceleración de la gravedad: $g = 9.8\ m / s^2$. Adaptado de[11] (Ramírez 2006).

```
/* Escriba, compile y ejecute un programa en C++ que calcule la altura máxima y el
alcance de un proyectil que describe un movimiento parabólico. */
#include <iostream>
#include <string>
#include <windows.h>
#include <locale.h>
#include <windows.h>
#define g 9.8
#define pi 3.1416
using namespace std;
int main()
{
   float angulo, veloin, xmax, hmax;
   string seguir = "S";
   while (seguir == "S" or seguir == "s")
   {
      system("cls");                //Comando de limpieza de la pantalla
      system("color 70");           //Comando de configuración de color
      setlocale(LC_ALL, "");
      cout << "\n\n\tPrograma que calcula el alcance y altura máxima de un
proyectil!\n";
      cout << "\t-------------------------------------------------------------
\n";
      Sleep(400);
      cout << "\n\n\tDigite el ángulo > 0 y < 90 y velocidad inicial del proyectil
en m/s\n\t";
      Sleep(400);
      cin >> angulo >> veloin;
      while (angulo <= 0 or angulo  >= 90 or veloin <= 0)        //Validación de
datos
      {
         cout << "\n\n\tError, digite el ángulo > 0 y < 90 y velocidad inicial del
proyectil en m/s\n\t";
         Sleep(400);
         cin >> angulo >> veloin;  //Lectura de los datos de entrada
      }
      xmax = pow(veloin, 2) * sin(2 * angulo * pi / 180) / g;
                                    //Cálculo delalcance máximo
```

11 *http://recursostic.educacion.es/descartes/web/materiales_didacticos/comp_movimientos/parabolico.htm*

```
        hmax = pow(veloin, 2) * pow(sin(angulo * pi / 180), 2) / (2 * g);
                                    //Cálculo de la altura máxima
        cout << "\n\n\t            Imprimir datos y resultados  \n";
        cout << "\t-------------------------------------------------\n";
        Sleep(400);
        cout << "\n\tEl ángulo de disparo del proyectil es:\t" << angulo << " grados"
<< endl;
        Sleep(400);
        cout << "\n\tLa velocidad inicial del proyectil es:\t" << veloin << " m/s" <<
endl;
        Sleep(400);
        cout << "\n\tEl alcance máximo del proyectil es:\t" << xmax << " m" << endl;
        Sleep(400);
        cout << "\n\tLa altura máxima del proyectil es:\t" << hmax << " m" << endl;
        Sleep(400);
        cout << "\n\t-------------------------------------------------\n";
        Sleep(400);
        cout << "\n\tDesea procesar otro proyectil S o N\t";
        cin >> seguir;
        while (seguir != "S" and seguir != "s" and seguir != "N" and seguir != "n")
        {
           cout << "\n\tError. Digite S o  N\t";
           Sleep(500); cin >> seguir;
        }
    }
    cout << endl << endl;
    cout << "\t";
    system("pause");
    return 0;
}
```

El resultado de la ejecución del programa se ilustra en la figura 7.

```
Programa que calcula el alcance y altura máxima de un proyectil!
----------------------------------------------------------------

Digite el ángulo > 0 y < 90 y velocidad inicial del proyectil en m/s
45     10

            Imprimir datos y resultados
-------------------------------------------------

El ángulo de disparo del proyectil es:  45 grados

La velocidad inicial del proyectil es:  10 m/s

El alcance máximo del proyectil es:     10.2041 m

La altura máxima del proyectil es:      2.55103 m

-------------------------------------------------

Desea procesar otro proyectil S o N
```

Figura 7. Resultado de la ejecución del programa que calcula el alcance y altura máxima de un proyectil. Fuente. Obtenido mediante (Visual Studio 2019).

Ejercicio 8

Escriba, compile y ejecute un programa en C++ que calcule la cantidad en gramos de cada nutriente, sí los carbohidratos aportan el 55% del valor calórico, los lípidos el 30% y las proteínas el 15%[12]. (Pfizer 2019) teniendo en cuenta que cada gramo de carbohidrato aporta 4 Kcal, de lípido 9 Kcal y proteína 4 Kcal[13], (Kidshealth 2018) para una dieta en Kcal que se lee por teclado para un paciente. Se debe mostrar el nombre del paciente, la dieta en Kcal y la cantidad de cada nutriente.

```
/* Escriba, compile y ejecute un programa en C++ que calcule la cantidad en gramos
de cada uno de los nutrientes para una dieta en Kcal que se lee por teclado. */
#include <iostream>
#include <string>
#include <locale.h>
#include <windows.h>
#define ec 0.55            //Se definen constantes del porcentaje calórico y energía
#define el 0.30
#define ep 0.15
#define kc 4
#define kl 9
#define kp 4
using namespace std;
int main()
{
   float gc, gl, gp, dieta, kcar, klip, kpro;
   string seguir = "S", nom;
   setlocale(LC_ALL, "");
   while (seguir == "S" or seguir == "s")
   {
      system("cls");         //Comando de limpieza de la pantalla
      system("color 70");    //Comando de configuración de color
      cout << endl;
      cout << "\n\n\tPrograma que calcula los gramos de nutriente para una dieta
calórica!\n";
      cout << "\t-----------------------------------------------------------------
---\n";
      cout << "\n\tDigite el nombre del paciente:\t";
      cin >> nom;
      cout << "\n\n\tDigite la dieta calórica en Kcal del paciente:\t";
```

12 *https://www.pfizer.es/salud/prevencion_habitos_saludables/dietas_nutricion/alimentacion_dieta_equilibrada.html*

13 *https://kidshealth.org/es/teens/fat-calories-esp.html*

```
      cin >> dieta;
      while (dieta <= 0)           //Validación de datos
      {
         cout << "\n\n\tError, digite valor mayor de 0, digite la dieta calórica en
Kcal\t";
         cin >> dieta;             //Lectura de los datos de entrada
      }
      kcar = ((float)dieta) * ec;   //Calculo requerimiento energetico
      klip = ((float)dieta) * el;
      kpro = ((float)dieta) * ep;
      gc = ((float)kcar) / kc;      //Calculo gramos de cada nutriente
      gl = ((float)klip) / kl;
      gp = ((float)kpro) / kp;
      cout << "\n\n\t                Imprimir datos y resultados          \n";
      cout << "\t-------------------------------------------------------------\n";
      cout << "\n\tEl nombre del paciente es:                 \t" << nom << endl;
      cout << "\n\tLa dieta calórica del paciente es:         \t" << dieta << "
Kcal" << endl;
      cout << "\n\tEl aporte calórico de los carbohidratos es:\t" << kcar << "
Kcal" << endl;
      cout << "\n\tEl aporte calórico de los lípidos es:      \t" << klip << "
Kcal" << endl;
      cout << "\n\tEl aporte calórico de las proteínas es:    \t" << kpro << "
Kcal" << endl;
      cout << "\n\tLos gramos de carbohidratos requeridos son:\t" << gc << " g" <<
endl;
      cout << "\n\tLos gramos de lípidos requeridos son:      \t" << gl << " g" <<
endl;
      cout << "\n\tLos gramos de proteínas requeridos son:    \t" << gp << " g" <<
endl;
      cout << "\n\t-------------------------------------------------------------\n";
      cout << "\n\tDesea procesar otro paciente S o N\t";
      cin >> seguir;
      while (seguir != "S" and seguir != "s" and seguir != "N" and seguir != "n")
      {
         cout << "\n\tError. Digite S o  N\t";
         Sleep(500); cin >> seguir;
      }
   }
   cout << endl << endl;
   cout << "\t";
   system("pause");
   return 0;
}
```

El resultado de la ejecución del programa se ilustra en la figura 8.

```
Programa que calcula los gramos de nutriente para una dieta calórica!
-----------------------------------------------------------------------

Digite el nombre del paciente:  Andrea

Digite la dieta calórica en Kcal del paciente:  2400

                 Imprimir datos y resultados
-------------------------------------------------------------
El nombre del paciente es:                      Andrea
La dieta calórica del paciente es:              2400 Kcal
El aporte calórico de los carbohidratos es:     1320 Kcal
El aporte calórico de los lípidos es:           720 Kcal
El aporte calórico de las proteínas es:         360 Kcal
Los gramos de carbohidratos requeridos son:     330 g
Los gramos de lípidos requeridos son:           80 g
Los gramos de proteínas requeridos son:         90 g
-------------------------------------------------------------

Desea procesar otro paciente S o N
```

Figura 8. Resultado de la ejecución del programa que calcula gramos requeridos de nutrientes para una dieta. Fuente. Obtenido mediante (Visual Studio 2019).

Ejercicio 9

Escriba, compile y ejecute un programa en C++ que calcule la longitud de un cilindro de aluminio conocido el radio, la masa y su densidad. Aplicar la fórmula $l = m / d * \pi * r^2$. Donde m es la masa, d la densidad del aluminio equivalente a 2.7 (g / cm ^{3}) [14], (Wikipedia 2020), r el radio del cilindro y π 3.1416.

```
/* Escriba, compile y ejecute un programa en C++ que calcule la longitud de un
cilindro de aluminio conocido el radio, la masa y su densidad. */
#include <iostream>
#include <string>
#include <locale.h>
#include <windows.h>
#define pi 3.1416
#define den 2.7
using namespace std;
int main()
```

14 *https://en.wikipedia.org/wiki/COVID-19_pandemic_in_Colombia*

```
{
   float mas, ra, lo;
   string seguir = "S";
   setlocale(LC_ALL, "");
   while (seguir == "S" or seguir == "s")
   {
      system("cls");            //Comando de limpieza de la pantalla
      system("color 70");       //Comando de configuración de color
      cout << endl;
      cout << "\n\tPrograma que calcula la longitud de un cilindro de aluminio!\n";
      cout << "\t-------------------------------------------------------------\n";
      cout << endl;
      cout << "\n\tCaptura y validación de datos";
      cout << "\n\t------------------------------\n\n";
      cout << "\n\tDigite la masa del cilindro en gramos:\t";
      cin >> mas;
      while (mas <= 0)
      {
         cout << "\n\tError, digite la masa del cilindro en gramos:\t";
         cin >> mas;
      }
      cout << "\n\tDigite el radio del cilindro en cm:\t";
      cin >> ra;
      while (ra <= 0)
      {
         cout << "\n\tError, digite el radio del cilindro en cm:\t";
         cin >> ra;
      }
      cout << "\n\tCalculando longitud del cilindro...";
      lo = (float(mas)) / (den * pi * pow(ra, 2));
      cout << endl << endl;
      cout << "\n\t       Imprimir datos y resultados       ";
      cout << "\n\t-----------------------------------------";
      cout << endl;
      cout << "\n\tMasa del cilindro en gramos:\t" << mas;
      cout << "\n\tRadio del cilindro en cm:\t" << ra;
      cout << "\n\tDensidad del aluminio g/cm^3:\t" << den;
      cout << "\n\tLongitud del cilindro en cm:\t" << lo;
      cout << "\n\t-----------------------------------------";
      cout << endl << endl;
      cout << "\n\tDesea procesar otro cilindro: S o  N\t";
      cin >> seguir;
      while (seguir != "S" and seguir != "s" and seguir != "N" and seguir != "n")
      {
         cout << "\n\tError. Digite S o  N\t";
         Sleep(500); cin >> seguir;
      }
   }
   cout << endl << endl;
```

```
    cout << "\t";
    system("pause");
    return 0;
}
```

El resultado de la ejecución del programa se ilustra en la figura 9.

```
Programa que calcula la longitud de un cilindro de aluminio!
------------------------------------------------------------

Captura y validación de datos
-----------------------------

Digite la masa del cilindro en gramos:  150
Digite el radio del cilindro en cm:     2
Calculando longitud del cilindro...

        Imprimir datos y resultados
-----------------------------------------
Masa del cilindro en gramos:    150
Radio del cilindro en cm:       2
Densidad del aluminio g/cm^3:   2.7
Longitud del cilindro en cm:    4.42096
-----------------------------------------

Desea procesar otro cilindro: S o  N
```

Figura 9. Resultado del cálculo de la longitud del cilindro de aluminio.
Fuente. Obtenido mediante (Visual Studio 2019).

Ejercicio 10

Escriba, compile y ejecute un programa en C++ que calcule la cantidad de calor Q que se debe aplicar a una masa de agua cuyo calor específico c.e. es 1 Cal / g * °C para elevar la temperatura en °C. Aplicar la fórmula Q = m * c.e * (Tf – Ti) en donde m representa la masa de agua, Tf es la temperatura final y Ti la inicial.

```
/* Escriba, compile y ejecute un programa en C++ que calcule la cantidad de calor Q
que se debe aplicar a una masa de agua para elevar la temperatura en °C. */
#include <iostream>
#include <string>
#include <windows.h>
#include <locale.h>
using namespace std;
int main()
{
```

```
float Q, mas, Tf, Ti;
string seguir = "S";
setlocale(LC_ALL, "");
while (seguir == "S" or seguir == "s")
{
   system("cls");            //Comando de limpieza de la pantalla
   system("color 70");       //Comando de configuración de color
   cout << endl;
   cout << "\n\tPrograma que calcula la cantidad de calor!\n";
   Sleep(400);
   cout << "\t-----------------------------------------\n";
   Sleep(400);
   cout << endl;
   cout << "\n\tCaptura y validación de datos";
   Sleep(400);
   cout << "\n\t-----------------------------\n\n";
   Sleep(400);
   cout << "\n\tDigite la masa de agua en gramos:  ";
   Sleep(400);
   cin >> mas;
   while (mas <= 0)
   {
      cout << "\n\tError, digite la masa de agua en gramos:  ";
      cin >> mas;
   }
   cout << "\n\tDigite la temperatura inicial en grados celsius:  ";
   Sleep(400);
   cin >> Ti;
   while (Ti <= -273)
   {
      cout << "\n\tError, digite la temperatura inicial en grados celsius:  ";
      Sleep(400);
      cin >> Ti;
   }
   cout << "\n\tDigite la temperatura final en grados celsius:  ";
   Sleep(400);
   cin >> Tf;
   while (Tf < Ti)
   {
      cout << "\n\tError, la temperatura final debe ser mayor que la inicial:  ";
      Sleep(400);
      cin >> Tf;
   }
   cout << "\n\tCalculando la cantidad de calor requerido...";
   Sleep(400);
   Q = mas * (Tf - Ti);
   cout << "\n\n\t              Imprimir datos y resultados            ";
   cout << "\n\t-------------------------------------------------------";
   cout << endl;
```

```
        cout << "\n\tMasa de agua en gramos:\t\t\t\t" << mas;
        Sleep(400);
        cout << "\n\tTemperatura inicial en grados celsius Ti:\t" << Ti;
        Sleep(400);
        cout << "\n\tTemperatura final en grados celsius Tf:\t\t" << Tf;
        Sleep(400);
        cout << "\n\tCantidad de calor en Calorías Q:\t\t" << Q;
        Sleep(400);
        cout << "\n\t-------------------------------------------------------";
        cout << "\n\n\tDesea procesar nuevamente: S o  N\t";
        Sleep(400);
        cin >> seguir;
        while (seguir != "S" and seguir != "s" and seguir != "N" and seguir != "n")
        {
           cout << "\n\tError. Digite S o  N\t";
           Sleep(500); cin >> seguir;
        }
    }
    cout << endl << endl;
    cout << "\t";
    system("pause");
    return 0;
}
```

El resultado de la ejecución del programa se ilustra en la figura 10.

```
Programa que calcula la cantidad de calor!
------------------------------------------

Captura y validación de datos
-----------------------------

Digite la masa de agua en gramos:  200
Digite la temperatura inicial en grados celsius:  -20
Digite la temperatura final en grados celsius:  50
Calculando la cantidad de calor requerido...

                Imprimir datos y resultados
-------------------------------------------------------

Masa de agua en gramos:                          200
Temperatura inicial en grados celsius Ti:        -20
Temperatura final en grados celsius Tf:          50
Cantidad de calor en Calorías Q:                 14000
-------------------------------------------------------

Desea procesar nuevamente: S o  N
```

Figura 10. Resultado de la cantidad de calor requerido para elevar la temperatura. Fuente. Obtenido mediante (Visual Studio 2019).